

Modbus RTU Driver V7 — Library for Coolmay FX3G PLC

Abstract

The Modbus RTU Driver enables a Coolmay FX3G PLC to operate as a Modbus RTU master or slave on its two auxiliary RS485 ports (port 2 and port 3). Communication channels are configured through an application-declared array `MB_CHANNELS` of type `MB_REG_50`; a single function block, `MB_PROCESS_50`, cycles through that array and issues the underlying `ADPRW` read/write requests on a **50 ms** tick.

Version V7 changes the channel-storage model: the `MB_CHANNELS` array and the `c_MB_CHANNELS_NUM` capacity constant are no longer declared by the library. The application declares them in its own global label list, sized to the number of channels it actually uses, and `MB_PROCESS_50` cycles through exactly that many channels. This keeps the automatically-assigned device usage proportional to the number of channels actually configured.

Prerequisites

1. The following libraries must be installed in the project before this driver:

- `Utils.sul`
- `TimeControl.sul`

2. The TimeControl library's `TCO_TICKER_50` ticker must be running. It advances at **50 ms** intervals and drives the internal scheduling of `MB_PROCESS_50`.

3. The application must declare the Modbus channel storage in its own global label list before using `MB_PROCESS_50`:

- `c_MB_CHANNELS_NUM` — `VAR_GLOBAL_CONSTANT` (`INT`), the upper bound (last valid index) of the channel array.
- `MB_CHANNELS` — `VAR_GLOBAL` (`ARRAY [0..c_MB_CHANNELS_NUM] OF MB_REG_50`), the channel array itself.

The literal array bound must match `c_MB_CHANNELS_NUM`. For three channels (indices 0–2), declare `c_MB_CHANNELS_NUM := 2` and `MB_CHANNELS : ARRAY [0..2] OF MB_REG_50`.

4. This library is compatible with **GX Works 2 v1.631H** and later. Updates are available from the `coolmay/soft` directory.

Terminology

- **Channel** — A single configured Modbus read/write relationship, described by one element of `MB_CHANNELS`.

- **Register channel** — A channel that transfers 16-bit register data (function codes **H3**, **H4**, **H6**, **H10**). Its value buffer lives in the **D** device area.
- **Coil channel** — A channel that transfers discrete bit data (function codes **H1**, **H2**, **H5**, **HF**). Its value buffer lives in the **M** device area.
- **Scratch buffer** — A **D**-area workspace used by **MB_PROCESS_50** during **ADPRW** transfers, starting at the **mb_iBuffer** base device.

Type names: this document uses the IEC type names written in the GX Works 2 label CSV files. The Label Editor displays them as follows:

IEC type	GX Works 2 Label Editor display
BOOL	Bit
INT	Word [Signed]
WORD	Word [Unsigned] / Bit String [16-bit]
DWORD	Double Word [Unsigned]

Architectural Description

This library enables a Coolmay FX3G PLC to operate as a **Modbus Slave** or a **Modbus Master** (on the secondary and tertiary RS485 ports — ports 2 and 3, respectively) for reading from and writing to Modbus RTU devices. It provides a low-overhead interface for configuring and managing Modbus communication channels.

The Modbus channels are described in a user-declared global array **MB_CHANNELS** of type **MB_REG_50**. The library does not fix the channel count: the application declares the array and the **c_MB_CHANNELS_NUM** upper bound in its global label list, and **MB_PROCESS_50** cycles through exactly that many channels.

Coolmay PLC/HMI integrated units are equipped with two RS485 ports: port 2 is exposed on the terminal connector, and port 3 is exposed on the DB9 connector. L02-series PLCs also provide two RS485 ports, both on terminal connectors.

The recommended workflow is:

1. At startup (under the **M8002** initialisation pulse), configure the timeout globals and the **MB_CHANNELS** array.
2. Initialise the required ports with **MB_PORT_SETTINGS** and the relevant **MB_*_INIT_*** function. Re-trigger the initialisation periodically after startup (see the timing note in § Modbus Master).
3. Call **fbMbProcess** every scan (or at least faster than the shortest **tCycle**).

Calling convention: the port-initialisation and settings POU's are *functions* that return their result through the function name. In GX Works 2 a function call used as a statement requires a left-hand side, so callers assign the result to a dummy bit, e.g. **M0 := MB_MASTER_INIT_PORT2(TRUE, PortSettings);**. The result value is not used elsewhere.

Function Blocks and Functions

Name	Type	Description
<code>MB_PORT_SETTINGS</code>	Function	Build the serial-port bit-field (parity, stop bits, baud rate).
<code>MB_MASTER_INIT_PORT2</code>	Function	Initialise the Modbus master on port 2.
<code>MB_MASTER_INIT_PORT3</code>	Function	Initialise the Modbus master on port 3.
<code>MB_SLAVE_INIT_PORT2</code>	Function	Initialise the Modbus slave on port 2.
<code>MB_SLAVE_INIT_PORT3</code>	Function	Initialise the Modbus slave on port 3.
<code>MTB_SLAVE_PORT2</code>	Function	Reconfigure port 2 for the Mitsubishi protocol.
<code>MTB_SLAVE_PORT3</code>	Function	Reconfigure port 3 for the Mitsubishi protocol.
<code>MB_PROCESS_50</code>	Function Block	Cycle through channels and issue <code>ADPRW</code> read/write requests.
<code>MB_REG_50</code>	Structure	Per-channel configuration (declared by the application as <code>MB_CHANNELS</code>).

Global Constants and Variables

Access Modes

Constant	Value	Description
<code>MB_READ_WRITE</code>	<code>0</code>	Read and write.
<code>MB_READ</code>	<code>1</code>	Read only.
<code>MB_WRITE</code>	<code>2</code>	Write only.

Clear-on-Start Modes

Constant	Value	Description
<code>MB_CLEAR_NONE</code>	<code>0</code>	Clear nothing (default).
<code>MB_CLEAR_ALL</code>	<code>1</code>	Clear both the value buffer and the change-tracking buffer.
<code>MB_CLEAR_BUFFER</code>	<code>2</code>	Clear the change-tracking buffer only.

Serial Port Settings

Constant	Value	Description
<code>MB_DL_7</code>	<code>0</code>	7 data bits.
<code>MB_DL_8</code>	<code>1</code>	8 data bits.
<code>MB_PARITY_NONE</code>	<code>0</code>	No parity.
<code>MB_PARITY_ODD</code>	<code>1</code>	Odd parity.

Constant	Value	Description
MB_PARITY_EVEN	2	Even parity.
MB_STOPBIT_1	0	1 stop bit.
MB_STOPBIT_2	1	2 stop bits.
MB_BPS_600	0	600 bps.
MB_BPS_1200	1	1200 bps.
MB_BPS_2400	2	2400 bps.
MB_BPS_4800	3	4800 bps.
MB_BPS_9600	4	9600 bps.
MB_BPS_19200	5	19200 bps.
MB_BPS_38400	6	38400 bps.
MB_BPS_57600	7	57600 bps.
MB_BPS_115200	8	115200 bps.

MB_DL_7 and MB_DL_8 are provided for completeness but are not selectable through MB_PORT_SETTINGS, which always configures **8 data bits**.

Ports

Constant	Value	Physical port
MB_PORT_2	0	RS485 port 2 — terminal connector (A, B)
MB_PORT_3	1	RS485 port 3 — DB9 connector (A1, B1)
MB_PORT_CAN	2	CAN port (H, L)
MB_PORT_TCP	3	Ethernet TCP port

MB_PORT_SETTINGS (Function)

Returns a correctly formatted bit-field value for initialising a port as either Master or Slave. The returned value is a **DWORD** bit-field for the port configuration register.

Variable	Scope	Type	Description
Parity	INPUT	INT	One of: MB_PARITY_NONE, MB_PARITY_ODD, MB_PARITY_EVEN.
StopBit	INPUT	INT	One of: MB_STOPBIT_1, MB_STOPBIT_2.
Baudrate	INPUT	INT	One of: MB_BPS_600, MB_BPS_1200, MB_BPS_2400, MB_BPS_4800, MB_BPS_9600, MB_BPS_19200, MB_BPS_38400, MB_BPS_57600, MB_BPS_115200.

Note: the data length is fixed at **8 data bits** and is not a parameter of this function.

Declare a local variable of type **DWORD** (e.g. **PortSettings**), then invoke the function:

```
VAR
    PortSettings : DWORD;
END_VAR

PortSettings := MB_PORT_SETTINGS(MB_PARITY_NONE, MB_STOPBIT_1, MB_BPS_9600);
```

Modbus Slave

MB_SLAVE_INIT_PORT2, MB_SLAVE_INIT_PORT3

These functions initialise the Modbus slave on port 2 or port 3 of the PLC, respectively.

Variable	Scope	Type	Description
xInit	INPUT	BOOL	Initialisation command. The slave is (re-)initialised on every rising edge.
iAddress	INPUT	INT	Modbus network address assigned to this PLC.
PortSettings	INPUT	DWORD	Return value of the MB_PORT_SETTINGS function.

Important — initialisation timing: the same caveat as for the master functions applies (see § Modbus Master). The function acts only on a rising edge of **xInit**, and triggering it solely from **M8002** may not reliably retain the port settings. Re-trigger it after startup, e.g. with the one-second clock **M8013**.

Example

The following snippet configures a slave with address **1** on port 2:

```
PortSettings := MB_PORT_SETTINGS(MB_PARITY_NONE, MB_STOPBIT_1, MB_BPS_9600);
M0 := MB_SLAVE_INIT_PORT2(M8013, 1, PortSettings);
```

No further configuration is necessary for the PLC to function as a slave on port 2. The same pattern applies to port 3. The variable **M0** is not referenced elsewhere in the program — it exists solely to satisfy the syntactic requirement that a function call used as a statement must have a left-hand side assignment.

Modbus Master

MB_MASTER_INIT_PORT2, MB_MASTER_INIT_PORT3

These functions initialise the Modbus Master on port 2 (terminal connector) or port 3 (DB9 connector), respectively.

Variable	Scope	Type	Description
<code>xInit</code>	INPUT	BOOL	Initialisation command. The master is (re-)initialised on every rising edge.
<code>PortSettings</code>	INPUT	DWORD	Return value of the <code>MB_PORT_SETTINGS</code> function.

Important — initialisation timing: The function acts only on a rising edge of `xInit`; re-initialising on every scan is **not required** and is **not performed**. Passing a constant `TRUE` fires the initialisation once, on the first scan. Triggering it solely from the `M8002` first-scan pulse is **not reliable**: the port configuration may not be retained because the settings can be overwritten later during startup. It is therefore recommended to re-trigger initialisation after startup. A common practice is to feed the one-second clock pulse `M8013` into `xInit`, so the port settings are re-written every second.

Example

```
PortSettings := MB_PORT_SETTINGS(MB_PARITY_NONE, MB_STOPBIT_1, MB_BPS_9600);
(* M8013 = 1 s clock: re-writes the port settings every second so they
   survive the startup overwrite. A constant TRUE fires only once. *)
M0 := MB_MASTER_INIT_PORT2(M8013, PortSettings);
```

Port Reconfiguration (Mitsubishi Protocol)

`MTB_SLAVE_PORT2`, `MTB_SLAVE_PORT3`

These functions reconfigure port 2 or port 3 to the Mitsubishi proprietary protocol (not Modbus RTU), set the station address to `1`, and set the serial format to **38400 baud, 7 data bits, even parity, 1 stop bit**. They are used to switch a port back from Modbus RTU to the Mitsubishi protocol.

Variable	Scope	Type	Description
<code>xInit</code>	INPUT	BOOL	Reconfiguration command. The port is reconfigured on every rising edge.

Example

```
M0 := MTB_SLAVE_PORT2(TRUE);
```

`MB_PROCESS_50` (Function Block)

This function block orchestrates all read and write operations across the configured channels. It must be called every scan (or at least faster than the shortest channel `tCycle`).

Prerequisite: The TimeControl library must be installed and `TCO_TICKER_50` must be running. This ticker advances at 50 ms intervals and drives the internal scheduling of channel operations.

Variable	Scope	Type	Description
<code>mb_xEnable</code>	INPUT	BOOL	Enables channel processing. When <code>FALSE</code> , the scheduler resets to its boot state.
<code>mb_iBuffer</code>	INPUT	INT	Base <code>D</code> device number of the scratch buffer used during <code>ADPRW</code> transfers. The scratch buffer holds up to <code>iNum</code> words for register channels, or $\lceil iNum / 16 \rceil$ words for coil channels (one bit per coil). It must not overlap any channel's value or change-tracking buffer.
<code>mb_iClearOnStart</code>	INPUT	INT	Startup buffer-clearing mode: <code>MB_CLEAR_ALL</code> , <code>MB_CLEAR_BUFFER</code> , or <code>MB_CLEAR_NONE</code> (default).
<code>mb_iTimeoutCount</code>	INPUT	INT	Consecutive timeouts before a channel is suspended. Default (when <code>0</code>): <code>2</code> .
<code>mb_iSuspendRetry</code>	INPUT	INT	Suspended-channel retry interval, in 50 ms units. Default (when <code>0</code>): <code>80</code> (4 s).
<code>mb_iTimeoutTime</code>	INPUT	INT	Timeout duration, in 50 ms units. Default (when <code>0</code>): <code>4</code> (200 ms).
<code>mb_iTimeout</code>	OUTPUT	INT	Channel number (index) of the channel that timed out. Held for one scan; <code>-1</code> when no channel has timed out.

Timeout Tuning

Call-interval consideration. The watchdog and the `ADPRW` completion flag (`M8029`) are only evaluated when `MB_PROCESS_50` is executed, so the *effective* resolution of `mb_iTimeoutTime` is the block's actual call interval — not the 50 ms the name implies. `mb_iTimeoutTime` remains specified in 50 ms units (`value × 50 ms`). Set it so the timeout lands at least one full call period past the expected request completion. Example: block called every 100 ms and a request expected to finish in 2 calls (200 ms) → use `mb_iTimeoutTime := 6` (300 ms), not `4` (200 ms); otherwise the completion and the timeout are sampled on the same boundary and the request can be falsely marked as timed out.

Clear-on-start. `mb_iClearOnStart` controls what the scheduler clears when it initialises the channel array (boot/enable). `MB_CLEAR_ALL` clears both the value buffer and the change-tracking buffer (the previous behaviour); `MB_CLEAR_BUFFER` clears only the change-tracking buffer, preserving the current value buffer; `MB_CLEAR_NONE` clears neither. Use `MB_CLEAR_NONE` or `MB_CLEAR_BUFFER` when values must be preserved across the startup reset.

The `MB_REG_50` Structure

The application declares a global array `MB_CHANNELS` of type `MB_REG_50`, together with a global constant `c_MB_CHANNELS_NUM` that holds the array's upper bound (last valid index). The library references both labels directly — it does not declare them. `MB_PROCESS_50` cycles through channels `0` to `c_MB_CHANNELS_NUM`, so the number of channels is fully under application control (e.g. `c_MB_CHANNELS_NUM := 2` for 3 channels, `:= 29` for 30 channels). Each channel may read or write up to **125** registers. The array must be configured once, typically on PLC startup under the `M8002` initialisation pulse flag.

The structure contains user-configurable fields (set by the application) and library-managed fields (maintained internally; do not modify).

User-Configurable Fields

Variable	Type	Description
<code>xEnabled</code>	BOOL	Enable this channel for processing. When <code>FALSE</code> , the channel is skipped.
<code>iDDevNum</code>	INT	Base device number of the value buffer. Register channels use <code>D</code> devices; coil channels use <code>M</code> devices. Each channel consumes <code>2 × iNum</code> devices. See § <code>iDDevNum</code> for allocation rules.
<code>iNum</code>	INT	Number of registers or coils to read/write (max 125). Default (when <code>0</code>): <code>1</code> .
<code>iReg</code>	WORD	Starting Modbus register/coil address. Declared <code>WORD</code> (unsigned) so addresses above 32,000 can be specified.
<code>iRF</code>	WORD	Modbus read function code (<code>H1–H4</code>). Default (when <code>0</code>): <code>H3</code> .
<code>iWF</code>	WORD	Modbus write function code (<code>H5, H6, HF, H10</code>). Default (when <code>0</code>): <code>H6</code> .
<code>iDev</code>	WORD	Modbus slave device address.
<code>tCycle</code>	INT	Cycle interval for automatic reads/writes, in units of 50 ms. E.g. <code>20</code> = 1 second. Set to <code>0</code> for manual-only operation via <code>xReadOnce</code> / <code>xWriteOnce</code> .
<code>iWR</code>	INT	Read/write mode: <code>MB_READ_WRITE</code> , <code>MB_READ</code> , or <code>MB_WRITE</code> . Writes (including <code>xWriteOnce</code> / <code>xWriteOnChange</code>) require <code>MB_READ_WRITE</code> or <code>MB_WRITE</code> .
<code>iPort</code>	INT	Communication port identifier: <code>MB_PORT_2</code> , <code>MB_PORT_3</code> , <code>MB_PORT_CAN</code> , or <code>MB_PORT_TCP</code> . See § Ports.
<code>xWriteOnChange</code>	BOOL	When <code>TRUE</code> , changed values are written to the slave immediately upon detection. When <code>FALSE</code> , writes occur only when the <code>tCycle</code> interval elapses.
<code>xReadOnce</code>	BOOL	On a rising edge (<code>FALSE</code> → <code>TRUE</code>), triggers a single read of this channel. For manual-only reads, set <code>tCycle</code> to <code>0</code> .
<code>xWriteOnce</code>	BOOL	On a rising edge (<code>FALSE</code> → <code>TRUE</code>), triggers a single write of this channel. For manual-only writes, set <code>tCycle</code> to <code>0</code> .
<code>xDone</code>	BOOL	Read-only completion pulse. Set <code>TRUE</code> on completion of one read/write cycle; cleared at the start of the next cycle. Primarily useful for channels operating in manual mode (<code>tCycle</code> = <code>0</code>).

Library-Managed Fields (Read-Only)

Variable	Type	Description
<code>tStart</code>	DWORD	Timestamp of the last completed operation (50 ms ticker).
<code>iTimeOut</code>	INT	Consecutive-timeout counter used for channel suspension. Reset to <code>0</code> on every successful poll.

Variable	Type	Description
<code>isRegister</code>	<code>BOOL</code>	<code>TRUE</code> = register channel, <code>FALSE</code> = coil channel. Derived from <code>iRF/iWF</code> .
<code>xReadOnceM</code>	<code>BOOL</code>	Rising-edge memory for <code>xReadOnce</code> .
<code>xWriteOnceM</code>	<code>BOOL</code>	Rising-edge memory for <code>xWriteOnce</code> .

Supported Read/Write Functions

Code	Mnemonic
<code>H1</code>	Read Coils
<code>H2</code>	Read Discrete Inputs
<code>H3</code>	Read Holding Registers
<code>H4</code>	Read Input Registers
<code>H5</code>	Write Single Coil
<code>H6</code>	Write Single Register
<code>HF</code>	Write Multiple Coils
<code>H10</code>	Write Multiple Registers

`iDDevNum` — Buffer Allocation

Each channel reserves **twice** the number of devices specified by `iNum`. Half of the allocation holds the actual register/coil values; the other half is used internally for change tracking. Register channels use the `D` device area; coil channels use the `M` device area.

If `iNum = 1` and `iDDevNum = 200`:

- `D200` stores the value.
- `D201` is reserved for internal use.

If `iNum = 5`:

- `D200–D204` store the values.
- `D205–D209` are reserved for internal use.

Caution: Failure to account for the internal buffer when assigning `iDDevNum` values across consecutive channels will result in data corruption.

Consider the following **incorrect** configuration:

```
MB_CHANNELS[0].iDDevNum := 600;
MB_CHANNELS[0].iNum := 3;
```

```
MB_CHANNELS[1].iDDevNum := 603;
MB_CHANNELS[1].iNum := 1;
```

Channel 0 consumes devices **D600** through **D605** (3 values + 3 internal). The first free device is therefore **D606** — not **D603**. Assigning **iDDevNum := 603** to channel 1 causes its buffer to overlap with the internal area of channel 0, producing undefined behaviour.

The **correct** allocation is:

```
MB_CHANNELS[0].iDDevNum := 600;
MB_CHANNELS[0].iNum := 3;

MB_CHANNELS[1].iDDevNum := 606;
MB_CHANNELS[1].iNum := 1;
```

iWF — Write Function Selection

Two write-function codes are available for register channels:

- **H6 (Write Single Register):** When assigned to a multi-register channel, only the register whose value changed is written in a dedicated request.
- **H10 (Write Multiple Registers):** When assigned to a multi-register channel, a change to any register within the group triggers a single request that updates the entire group.

Guidance:

- For double-word values spanning multiple registers → use **H10**.
- For multiple independent single-register variables on the same channel → use **H6**.
- For a single-register channel → use **H6**.

The same principle applies to coil channels: use **H5** (Write Single Coil), even when the channel spans multiple coils.

Complete Example

Before configuring channels, declare the storage in the project's global label list. The following declares three channels (indices **0–2**):

Label	Class	Type / Value
c_MB_CHANNELS_NUM	VAR_GLOBAL_CONSTANT	INT = 2
MB_CHANNELS	VAR_GLOBAL	ARRAY [0..2] OF MB_REG_50

Declare the function-block instance and the port bit-field in the local label section of the POU:

```
VAR
    fbMbProcess : MB_PROCESS_50;
```

```
PortSettings : DWORD;  
END_VAR
```

M0, **M1**, **M2**, and **M8002** in the example are device labels (auxiliary relays and the initialisation-pulse special relay) and do not require a **VAR** declaration.

```
IF M8002 THEN
```

```
    (* Timeout and clear-on-start tuning are supplied to fbMbProcess below. *)  
    PortSettings := MB_PORT_SETTINGS(MB_PARITY_NONE, MB_STOPBIT_1, MB_BPS_9600);
```

```
    (* Automatic read/write channel on port 2 *)  
    MB_CHANNELS[0].xEnabled := TRUE;  
    MB_CHANNELS[0].iDDevNum := 600;  
    MB_CHANNELS[0].iNum := 3;  
    MB_CHANNELS[0].iReg := K16384;  
    MB_CHANNELS[0].iRF := H3;  
    MB_CHANNELS[0].iWF := H6;  
    MB_CHANNELS[0].iDev := H1;  
    MB_CHANNELS[0].tCycle := 20; (* 20 x 50 ms = 1 000 ms = 1 second *)  
    MB_CHANNELS[0].xWriteOnChange := TRUE;  
    MB_CHANNELS[0].iWR := MB_READ_WRITE;  
    MB_CHANNELS[0].iPort := MB_PORT_2;
```

```
    (* Minimal setup – connects to a device via Port 3 *)  
    MB_CHANNELS[1].xEnabled := TRUE;  
    MB_CHANNELS[1].iDDevNum := 610;  
    MB_CHANNELS[1].iReg := K16385;  
    MB_CHANNELS[1].iDev := K16;  
    MB_CHANNELS[1].tCycle := 4;  
    MB_CHANNELS[1].iWR := MB_READ;  
    MB_CHANNELS[1].iPort := MB_PORT_3;
```

```
    (* Manual-cycle read/write of a 3-register group *)  
    MB_CHANNELS[2].xEnabled := TRUE;  
    MB_CHANNELS[2].iNum := 3;  
    MB_CHANNELS[2].iDev := H1;  
    MB_CHANNELS[2].iPort := MB_PORT_3;  
    MB_CHANNELS[2].iDDevNum := 10;  
    MB_CHANNELS[2].iReg := H1000;  
    MB_CHANNELS[2].iRF := H3;  
    MB_CHANNELS[2].iWR := MB_READ_WRITE;  
    MB_CHANNELS[2].tCycle := 0;
```

```
END_IF;
```

```
    (* Multi-master mode is available on both ports for L02-series PLCs.  
    M8013 = 1 s clock re-writes the port settings every second so they  
    survive the startup overwrite. *)  
    M0 := MB_MASTER_INIT_PORT2(M8013, PortSettings);  
    M0 := MB_MASTER_INIT_PORT3(M8013, PortSettings);
```

```

fbMbProcess(mb_xEnable := TRUE, mb_iBuffer := 100,
  mb_iClearOnStart := MB_CLEAR_ALL,
  mb_iTimeoutCount := 2,
  mb_iSuspendRetry := 80,
  mb_iTimeoutTime := 4);

(* Manually read channel 2 once *)
IF M1 THEN
  MB_CHANNELS[2].xReadOnce := TRUE;
  IF MB_CHANNELS[2].xDone = TRUE THEN
    M1 := FALSE;
    MB_CHANNELS[2].xReadOnce := FALSE;
  END_IF;
END_IF;

(* Manually write channel 2 once *)
IF M2 THEN
  D10 := 100;
  D11 := 100;
  D12 := 100;
  MB_CHANNELS[2].xWriteOnce := TRUE;
  IF MB_CHANNELS[2].xDone = TRUE THEN
    M2 := FALSE;
    MB_CHANNELS[2].xWriteOnce := FALSE;
  END_IF;
END_IF;

```

Post-configuration behaviour:

- **D600**, **D601**, and **D602** hold the three register values read from device address **1**, refreshed every second. If any value changes locally, the slave is updated immediately (**xWriteOnChange := TRUE**).
- **D610** holds the single register value read from device address **16**. Since this channel is configured as read-only (**MB_READ**), any local modification to **D610** is overwritten by the next read cycle (every 200 ms).
- Channel 2 (**tCycle = 0**) performs no automatic transfers. It is read or written only when **xReadOnce** / **xWriteOnce** is pulsed.

Alternative Write Confirmation

The expression **IF D10 = D13 AND D11 = D14 AND D12 = D15 THEN** implements a buffer-comparison strategy for verifying write completion.

As described in § **iDDevNum**, each channel reserves twice the number of devices specified by **iNum**. With **iDDevNum := 10** and **iNum := 3**:

- **D10–D12** hold the actual register values.
- **D13–D15** serve as the change-tracking buffer.

Upon a successful write, the library updates the change-tracking buffer to match the newly written values. Therefore, equality between the value registers and their corresponding change-tracking registers confirms that the write has completed.

```

(* Alternative write-confirmation technique *)
IF M2 THEN
    D10 := 200;
    D11 := 200;
    D12 := 200;
    MB_CHANNELS[2].xWriteOnce := TRUE;
    IF D10 = D13 AND D11 = D14 AND D12 = D15 THEN
        M2 := FALSE;
        MB_CHANNELS[2].xWriteOnce := FALSE;
    END_IF;
END_IF;

```

This technique is particularly useful when employing `xWriteOnChange` (rather than `xWriteOnce`) with a multi-register channel and write function `H6`. If, for example, two registers within a ten-register channel are modified, `H6` writes each one individually, setting `xDone` once per successful write. Counting two distinct `xDone` pulses can rapidly become unwieldy. In contrast, a single comparison of the form `IF D10 = D13 AND D11 = D14 AND ...` confirms that all changed registers have been successfully propagated to the slave in a single check.

Timeout and Suspension Mechanism

The library implements a channel-suspension policy for fault tolerance. If a channel fails to receive a response for `mb_iTimeoutCount` consecutive attempts, it is flagged as **suspended**. Once suspended, the channel is polled at a reduced rate — once every `mb_iSuspendRetry` interval. As soon as a valid response is received, the suspension flag is cleared and the channel resumes its normal cycle interval as defined by `MB_CHANNELS[*].tCycle`.

When an `ADPRW` request times out, `MB_PROCESS_50.mb_iTimeout` reports the index of the failing channel (0-based) for one scan and is `-1` otherwise. This output can be used to react to a specific channel timing out, e.g. to latch an alarm or log the event.

The watchdog timer is evaluated **after** the request handling, so a request whose completion (`M8029`) arrives in the same block call as the timeout deadline is counted as a success (the `iTimeout` counter is reset and the snapshot is updated) rather than as a timeout. Set `mb_iTimeoutTime` comfortably longer than the expected request + slave-response round-trip time and account for the block's call interval (see Timeout Tuning above); if it is too short every request is abandoned as timed out before its `M8029` arrives, the counter only ever increments, and the channel enters the suspend/retry loop without ever clearing.